MINI PROJECT • DEEP LEARNING

Image Colorization
ภาพวาดสเก็ต → เติมสีให้ภาพอัตโนมัติ

การเรียนรู้เชิงลึกเพื่อการแปลงภาพลายเส้นเป็นภาพสีเป้าหมายด้วย U-Net

สมาชิกกลุ่มคนที่ 1

นายสรรพัญญู (Sapphanyu)

รหัสนักศึกษา: [กรอกรหัสสมาชิกคนที่ 1]

สมาชิกกลุ่มคนที่ 2

[ชื่อ-นามสกุล สมาชิกคนที่ 2]

รหัสนักศึกษา: [กรอกรหัสสมาชิกคนที่ 2]

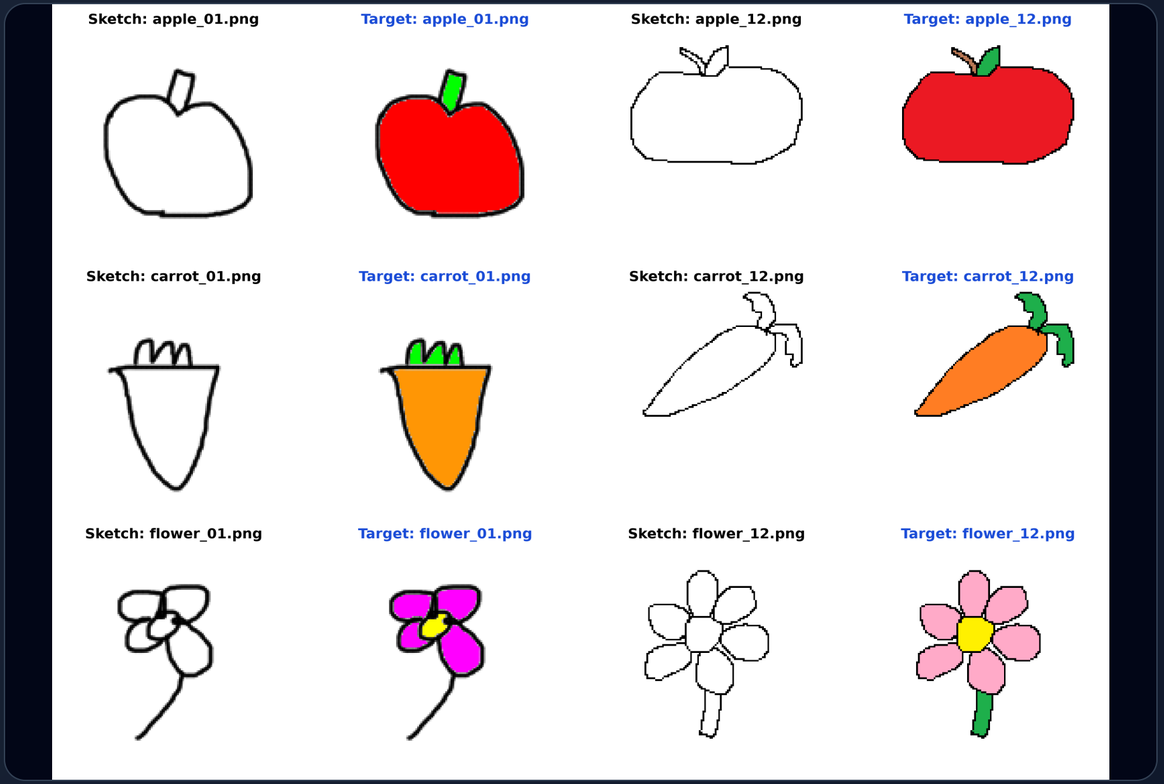
1. หัวข้อภาพและการสร้างชุดข้อมูล (Dataset)

การวาดภาพสเก็ตและสร้างภาพสีเป้าหมายด้วยตนเอง 100%

Data Creation

🎨 หัวข้อภาพที่เลือกทำ

- **หมวดหมู่วัตถุธรรมชาติ 3 ชนิด:**
  1. **Apple** ผลแอปเปิ้ล (20 คู่ภาพ)
  2. **Carrot** หัวแครอท (20 คู่ภาพ)
  3. **Flower** ดอกไม้ (20 คู่ภาพ)
- **การแบ่งงานในกลุ่ม (2 คน):**
  - สมาชิกคนที่ 1: สร้าง 30 ภาพสเก็ต + 30 ภาพสี
  - สมาชิกคนที่ 2: สร้าง 30 ภาพสเก็ต + 30 ภาพสี
- **รวมทั้งหมด:** 60 คู่ภาพ (120 ไฟล์) แบบ Paired Images
- **คุณลักษณะภาพ:** มีความหลากหลายของเส้น ลวดลาย และเฉดสี เพื่อท้าทายโมเดล



ตัวอย่างคู่ภาพ (Sketch ลายเส้น ซ้าย | Color สีเป้าหมาย ขวา)

2. การเตรียมข้อมูลและการแบ่ง Train / Test Split

Data Pipeline, Resizing, Normalization และสัดส่วนการทดลอง

Preprocessing

⚙️ การเตรียมข้อมูล (Preprocessing)

- **Standardize Resolution:** ปรับขนาดทุกภาพเป็น `128 × 128 px`
- **Color Mode:** แปลงเป็นภาพ RGB 3 Channels
- **Normalization:** ปรับค่าความสว่างพิกเซลจาก `[0, 255]` ให้อยู่ในช่วง `[-1, 1]` เพื่อให้สอดคล้องกับ Activation Tanh ที่ Output layer
- **Paired Filename Matching:** จัดชื่อไฟล์ตรงกัน 1:1 เช่น `apple_01.png`

📊 การแบ่งชุดข้อมูล (Train / Test Split)

- **สัดส่วนการแบ่ง: 80% : 20%**
  - **Train Set: 80% (48 คู่ภาพ)** → ใช้ฝึกสอนโมเดล
  - **Test Set: 20% (12 คู่ภาพ)** → ใช้วัดประสิทธิภาพอย่างเป็นธรรม (ภาพ unseen)
- **Reproducibility:** ล็อกค่า `np.random.seed(42)` ทำให้ผลการสุ่มคงที่ สามารถรันซ้ำได้ 100%
- **Batch Size:** กำหนดขนาด Batch = 8 ผ่าน PyTorch DataLoader

3. สถาปัตยกรรมโมเดล (U-Net Architecture)

โครงสร้างโมเดล Image-to-Image พร้อม Skip Connections และ Loss Function

Deep Learning Model

🧠 ทำไมจึงเลือกโมเดล U-Net?

- **Encoder (Contracting Path):** ใช้ Convolution 3x3 + BatchNorm + ReLU สกัด Feature และลดมิติด้วย MaxPool2d (32 ➔ 64 ➔ 128 ➔ 256 channels)
- **Decoder (Expansive Path):** ทำ Upsampling ด้วย ConvTranspose2d ขยายมิติกลับสู่ 128x128
- **Skip Connections (จุดเด่นหลัก):** เชื่อมโยงข้อมูลลายเส้นจาก Encoder ไปยัง Decoder โดยตรง ช่วยให้**ขอบภาพคมชัด ลายเส้นไม่เบลอมัว**
- **Output Layer:** Conv 1x1 + Tanh ผลิตภาพสี RGB 3 ช่อง

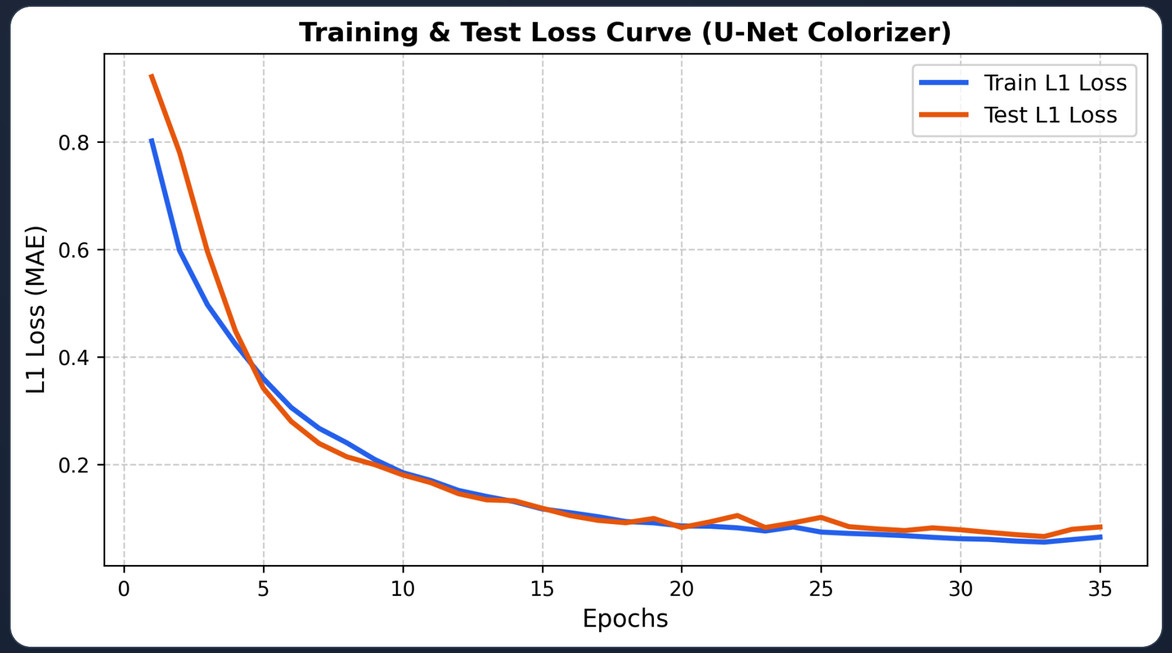
🎯 ฟังก์ชันเป้าหมายและการวัดประสิทธิภาพ

- **Loss Function: L1 Loss (MAE)**
  เลือกใช้ L1 Loss แทน L2 (MSE) เพราะช่วยลดปัญหาภาพเบลอ (Blurriness) และให้สีสด คมชัดตรงตามเป้าหมายมากกว่า
- **Optimizer:** Adam Optimizer ด้วย Learning Rate = `0.001`
- **เกณฑ์การวัดประสิทธิภาพ (Evaluation Metrics):**
  1. **MAE (Mean Absolute Error):** วัดค่าความคลาดเคลื่อนเฉลี่ยของระดับสี
  2. **PSNR (Peak Signal-to-Noise Ratio):** วัดคุณภาพและความคมชัดของภาพ (dB)

4. ผลการฝึกสอนและการวัดประสิทธิภาพ (Evaluation)

กราฟการลู่เข้าของ Loss Curve และตัวเลขชี้วัดบนชุดทดสอบ (Test Set)

Performance Metrics



📈 ผลการเทรน (Training Analysis)

- **การลู่เข้าของโมเดล (Convergence):**
  - Train Loss ลดลงอย่างมีนัยสำคัญจาก **0.8010 ➔ 0.0647**
  - Test Loss ลดลงสอดคล้องกันจาก **0.9204 ➔ 0.0835**
- **ไม่มีภาวะ Overfitting:** กราฟ Train และ Test Loss ลดลงไปในทิศทางเดียวกันอย่างมีเสถียรภาพ

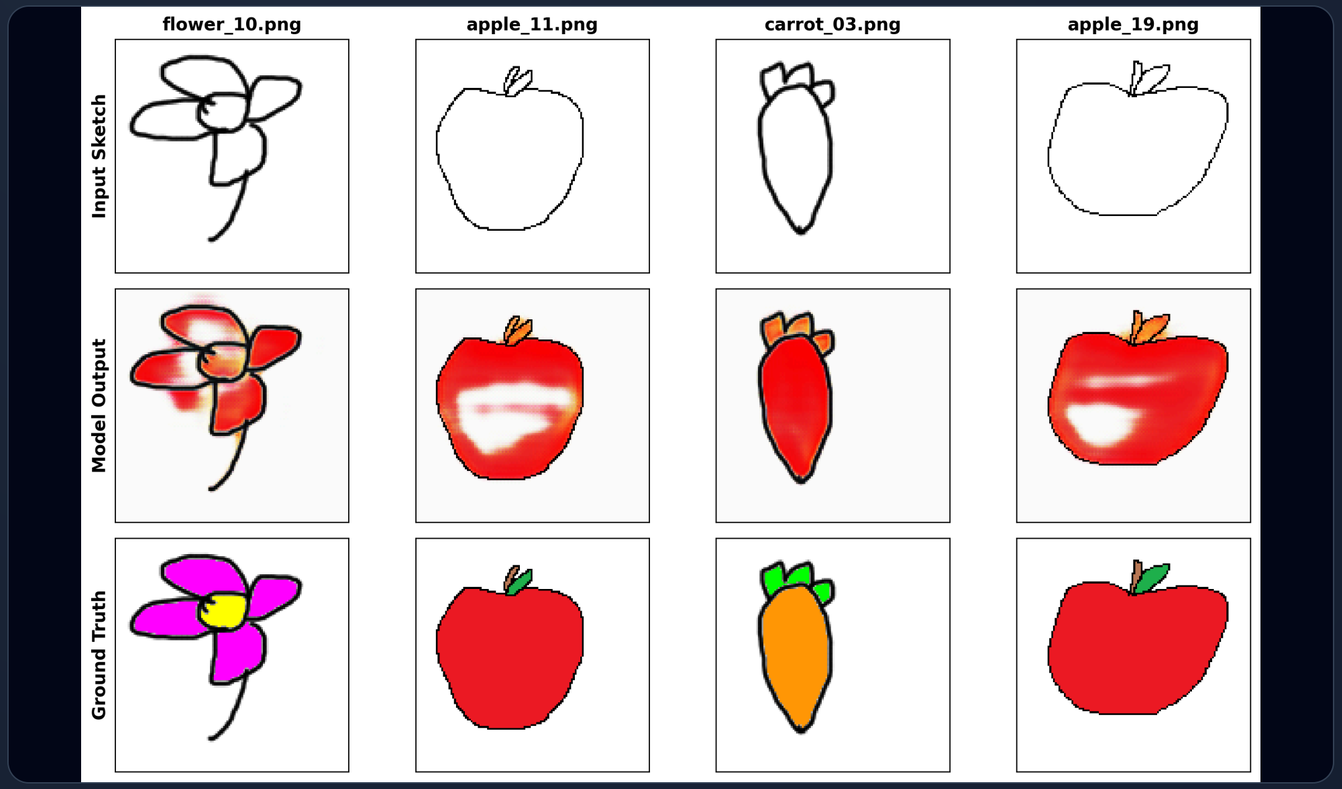
0.0418
Average Test MAE (ยิ่งน้อยยิ่งดี)

18.39 dB
Average Test PSNR (ความคมชัด)

5. ผลลัพธ์ภาพที่ได้และข้อสรุป (Qualitative Results)

เปรียบเทียบภาพวาดสเก็ต ➔ ภาพสีที่โมเดลเติมให้ ➔ ภาพเป้าหมายจริง (Test Set)

Results & Summary



🔍 การวิเคราะห์ผลลัพธ์ (Discussion)

- **ความแม่นยำของสี (Color Fidelity):** โมเดลสามารถจดจำรูปทรงของผลไม้และดอกไม้ แล้วเลือกใช้สีที่สอดคล้องกับธรรมชาติได้ถูกต้อง (แอปเปิ้ลสีแดง, แครอทสีส้ม, ใบไม้สีเขียว)
- **ความคมชัดของลายเส้น:** ไม่เกิดการเบลอหรือสีรั่วออกจากกรอบลายเส้นอันเนื่องมาจากประสิทธิภาพของ Skip Connections
- **สรุปผลโครงงาน:** U-Net ตอบโจทย์งาน Image Colorization ได้อย่างมีประสิทธิภาพแม้มีข้อจำกัดด้านขนาดข้อมูลขนาดกะทัดรัด